



Figure 2: The reverse scalability graph shows various job numbers on a kernel build on a quad core machine

Then reboot your machine to the new kernel. You will get a new kernel entry in Grub that looks like "Ubuntu 9.04, kernel 2.6.31-bfs211-bfskernel". Log in and experience the difference.

Performance analysis

As Kolivas explains in his FAQ page, it is not the number of physical CPUs that affect BFS's scalability. Rather, it's that of logical ones. This means that a hyper-threaded quad core CPU is simulated as 8 logical CPUs.

People are also concerned about the $O(n)$ lookup of BFS. However, the actual overhead is very small. Even with large numbers of n , it has lower overhead than an $O(1)$ design. During a test against CFS, in which the test application forks 1,000 simple tasks, it showed no difference in time as compared to CFS on a quad core machine. But BFS will be much faster when the load is lower and the CPUs are less in number, which is normally the case with desktops.

Kolivas cites an example in the FAQ page, "See the reverse scalability graph, courtesy of Serge Belyshev, showing various job numbers on a kernel build on a quad core machine [Figure 2]. The problem has always been that the mainline scheduler can't keep the CPUs busy enough; i.e., it doesn't make the most of your hardware in the most common situations on a desktop! Note that the reverse scalability graph is old; the scalability has improved since then."

To test the speed of the new scheduler, I compiled Python 2.6.2 with a `make -j2` on my laptop that has a Core2Duo processor and 2GB of RAM. It took only 1 minute and 34 seconds to complete on the new BFS patched kernel. The compilation using simple `make` (without the `-jX` option) completed in 2 minutes and 2 seconds.

Earlier, I had done this test on the stock kernel that uses CFS. But it took 1 minute and 41 seconds using `make -j2` and 2 minutes and 4 seconds without using the `-jX` option to compile the same version of Python. So the speed difference is obvious. Refer to Tables 1 and 2 for the time differences.

make -j2 performance while compiling Python 2.6.2

	when using BFS	when using CFS
real	1 minute, 34.297 seconds	1 minute 41.841 seconds
user	1 minute, 56.151 seconds	1 minutes 57.387 seconds
sys	7.836 seconds	8.309 seconds

Table 1

make performance while compiling Python 2.6.2

	when using BFS	when using CFS
real	2 minutes, 2.595 seconds	2 minutes, 4.598 seconds
user	1 minute, 54.183 seconds	1 minute, 54.519 seconds
sys	7.576 seconds	7.836 seconds

Table 2

Some users from the Ubuntu Forums reported that they have experienced a speed difference in GDM. "Before the patch, there was a period of some five seconds during which GDM seemed to be in deep thought mode, trying to decide which users to show in this list. After the patch the list appeared almost immediately."

There have been only positive reviews of the BFS scheduler throughout the Linux community, as far as I could see. As the developer says, BFS is not stable yet and when it reaches a stable release, I'm pretty sure it will have something more to offer. By the way, Kolivas concludes his FAQ page by pointing out, "Yes I know you probably still can't watch full screen videos on YouTube, but that's not entirely the scheduler's fault." 

References

- Wikipedia on schedulers: [en.wikipedia.org/wiki/Scheduling_\(computing\)](http://en.wikipedia.org/wiki/Scheduling_(computing))
- Types of scheduling: <http://www.cim.mcgill.ca/~franco/OpSys-304-427/lecture-notes/node38.html>
- Wikipedia on CFS: en.wikipedia.org/wiki/Completely_Fair_Scheduler
- BFS FAQ: ck.kolivas.org/patches/bfs/bfs-faq.txt
- Con Kolivas interview: ck.kolivas.org/german_interview_magazine_interview.txt
- The reverse scalability graph is taken from ck.kolivas.org/patches/bfs/reverse-scalability.png

By: Sreejith K.

The author is a FOSS enthusiast and a hardcore Ubuntu fan who loves hacking GNU/Linux. He's currently working as a development engineer at K7 Computing, Chennai. His areas of interest include compilers, virtualisation technologies and cloud computing. He blogs at <http://semkin.in> and can be reached at sreejithenka@gmail.com.